

Chapter 9: Population Methods

Algorithms for Optimization, 2nd Edition (2026)

Kochenderfer & Wheeler

Adapted for Lecture Slides

2026

Chapter Outline

- 1 Population Iteration
- 2 Genetic Algorithms
- 3 Differential Evolution
- 4 Particle Swarm Optimization
- 5 Firefly Algorithm
- 6 Cuckoo Search
- 7 Hybrid Methods
- 8 Summary & Exercises

Motivation: Why Use a Population of Solutions? I

All the optimization methods covered in previous chapters share one structural feature: they maintain a *single* candidate design point $\mathbf{x}$ and iteratively improve it. This single-point strategy works well for smooth, unimodal objectives — that is, functions with exactly one minimum and no misleading valleys — but it has a fundamental weakness: once the search converges to a local minimum, there is no information anywhere else in the design space to draw on, and the algorithm simply stops improving.

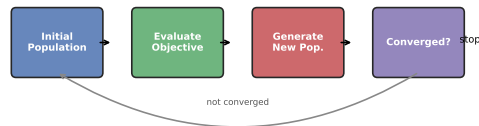
Population methods take a fundamentally different approach. Instead of tracking one design, they maintain a *collection* (a “population”) of m (m) candidate solutions $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(m)}$ and evolve all of them simultaneously through many iterations called *generations*. Here, $\mathbf{x}^{(i)}$ ($\mathbf{x}$ superscript i in round brackets) denotes the i -th (i -th) *individual* in the population, and the superscript in round brackets distinguishes it from an exponent.

Four Key Advantages of Population Methods

1. Information sharing across the design space.

Because different individuals occupy different regions of the design space simultaneously, the algorithm can detect structure such as ridges, valleys, and flat plateaus that a single-point method exploring sequentially would never observe. Individuals from different regions can exchange

Population Iteration Loop



The fundamental population loop: initialise m individuals, evaluate the objective f at each, generate

Motivation: Why Use a Population of Solutions? II

Key Insight: Diversity versus Convergence

Every population method faces a fundamental tension between *exploration* (keeping individuals spread across the design space to avoid missing the global optimum) and *exploitation* (concentrating individuals near the current best solutions to refine them efficiently). Too much exploration wastes evaluations on unproductive regions; too much exploitation leads to premature convergence at a local minimum. The methods in this chapter each resolve this tension in a different way, and understanding that trade-off is the key to choosing the right method for a given problem.

Population Iteration: The General Template I

Before studying specific algorithms (genetic algorithms, differential evolution, particle swarm optimization, etc.), it is helpful to see that they all share the same high-level algorithmic skeleton. Understanding this shared structure makes it easy to switch between methods: only one step changes.

Population Iteration: The General Template II

General Population Method — Algorithm 9.1

- ➊ **Initialise** the population $\mathcal{P} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$, typically by drawing each individual uniformly at random from the feasible design space (the set of all allowed design vectors). This ensures no region of the space is systematically excluded from the initial search.
- ➋ For $k = 1, 2, \dots, k_{\max}$ (k sub max, the maximum number of generations, a user-specified budget):
 - Ⓐ Evaluate $f(\mathbf{x}^{(i)})$ for every individual i in the current population, where f is the objective function we are trying to minimise.
 - Ⓑ **Generate** a new population using a method-specific rule. This rule typically rewards lower-objective individuals (selection), combines information from multiple parents (crossover or attraction), and introduces randomness (mutation or random walks). *This is the only step that differs between methods.*
- ➌ Return the best individual found across all generations: $\mathbf{x}^* = \arg \min_i f(\mathbf{x}^{(i)})$, where $\arg \min$ (“argmin”, meaning “the argument that minimises”) extracts the individual with the smallest objective value.

Here $k_{\max}$ (k sub max) is typically set by specifying a total function evaluation budget. Since each generation performs m (m) evaluations (one per individual), the total number of evaluations is $m \times k_{\max}$.

Population Iteration: The General Template III

Worked Example: Counting Evaluations

Suppose we run a population method with $m = 50$ individuals and $k_{\max} = 100$ generations. The total number of objective function evaluations is $50 \times 100 = 5,000$. If each evaluation takes 0.1 seconds, the total wall-clock time is $5,000 \times 0.1 = 500$ seconds ≈ 8.3 minutes. If the evaluations can be parallelised across 50 CPU cores, the time reduces to $100 \times 0.1 = 10$ seconds — a 50-fold speed-up. This illustrates why population methods are so attractive for computationally expensive objectives.

Key Insight: One Skeleton, Many Algorithms

The only part that differs between genetic algorithms, differential evolution, PSO, and other methods in this chapter is *step 2(b)*: how the new population is generated from the current one. The initialisation and termination logic are identical. When you encounter a new population method, your first task should be to identify exactly what rule it uses in step 2(b).

Population Iteration: Python Skeleton

```
1 import numpy as np
2
3 def population_method(f, bounds, pop_size=50, k_max=100, rng=None):
4     """Generic population loop. Override 'step' to specialise."""
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T          # lower and upper bounds per dimension
7     # Step 1: uniform initialisation across the feasible space
8     population = rng.uniform(lo, hi, (pop_size, len(lo)))
9     for _ in range(k_max):
10         # Step 2a: evaluate objective at every individual
11         y = np.array([f(x) for x in population])
12         # Step 2b: method-specific update (returns new population of same size)
13         population = step(population, y, rng)
14     # Return best individual found
15     y = np.array([f(x) for x in population])
16     return population[np.argmin(y)]
```

Listing 1: Generic population method skeleton (Python / NumPy)

In this skeleton, `bounds` is a list of `(lo, hi)` pairs, one per design variable dimension. For example, if the design space is the square $[-3, 3]^2$, then `bounds = [(-3, 3), (-3, 3)]`. The array `population` has shape `(pop_size, n)`, where `pop_size` is m (m) and n is the number of design variables. The function `step` — which you override when implementing a specific method — takes the current population matrix and its objective values and returns a new population matrix of the same shape. To implement a GA you replace `step` with `selection + crossover + mutation`; for DE you replace it with the

Genetic Algorithms: Biological Inspiration and Overview I

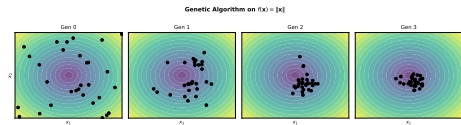
Genetic Algorithms (GAs) are the oldest and most well-known class of population methods. They were developed by John Holland in the 1970s and draw their inspiration from Darwinian natural selection: individuals that are better adapted to their environment survive and reproduce more often, gradually shifting the population toward higher fitness over many generations.

In the optimization context, “fitness” corresponds to having a low objective value $f(\mathbf{x})$ (since we are minimising, a lower objective is “better” just as a better-adapted organism survives more easily). Each individual $\mathbf{x}$ is called a *chromosome*, and each component x_j (x sub j) of the design vector is called a *gene*. Genes may be real-valued (for continuous design spaces) or binary (for combinatorial problems).

Three Core Operators — Algorithm 9.2

A GA applies three operators each generation in sequence:

- 1 **Selection** — decide which individuals get to “reproduce” (become parents of the next generation), favouring those with lower objective values so that useful gene combinations are propagated forward.
- 2 **Crossover** (also called *recombination*) — combine the chromosomes (design vectors) of two selected parent individuals to create offspring. This is the mechanism by



Four successive generations of a genetic algorithm minimising $f(\mathbf{x}) = \|\mathbf{x}\|$ (the Euclidean distance from the origin, i.e., $\sqrt{x_1^2 + x_2^2}$, whose minimum is 0 at the origin). The population visibly concentrates toward $\mathbf{x} = \mathbf{0}$ (the global minimum) as generations pass.

Notation: The Fitness Conversion Formula

In the formula above:

- $\max\{f(\mathbf{x}^{(1)}), \dots, f(\mathbf{x}^{(m)})\}$ is the largest (worst) objective value among all m (m) individuals in the current population.
- $f(\mathbf{x}^{(i)})$ is the objective value of individual i .
- fitness_i (fitness sub i) is the resulting fitness score for individual i .

Subtracting the individual's objective from the maximum gives a non-negative number: the worst individual (with the highest objective) gets $\text{fitness} = 0$, while the best individual (with the lowest objective) gets the highest fitness value. All other individuals lie in between.

How to Read This Formula

Think of it this way: if the worst objective in the population is 10 and an individual has objective 3, its fitness is $10 - 3 = 7$. Another individual with objective 9 gets fitness $10 - 9 = 1$. The individual with fitness 7 is seven times more attractive to roulette-wheel selection than the individual with fitness 1. As the population improves over generations, the maximum objective shrinks, and so do the fitness values — a natural self-calibration that keeps selection pressure stable.

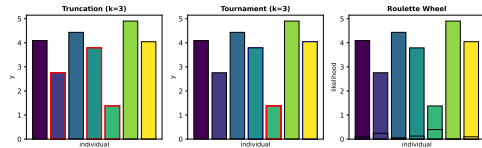
9.2.1 Selection: Choosing Which Individuals Reproduce I

Selection is the mechanism by which a genetic algorithm applies “survival of the fittest”. The goal is to choose parent individuals for the next generation so that individuals with lower objective values (higher fitness) are more likely to pass their genes on, while still allowing some less-fit individuals a chance to reproduce (to preserve diversity). Three common selection methods are described below.

Method 1: Truncation Selection

Keep only the k (k , the truncation size) best individuals — those with the k lowest objective values — and discard all others. New parent pairs are sampled uniformly at random from this elite subset of size k . Conceptually: sort the population by objective value, take the top k , and sample from those. The parameter k is typically set to around 10% to 20% of the population size (for example, $k = 10$ out of $m = 100$).

The trade-off is that truncation can rapidly reduce genetic diversity, since only k unique parent chromosomes are available. If k is too small, the population can prematurely collapse to a single local minimum.



Left: Truncation selection — the three individuals with highlighted borders are the top- k elite; only these are used as parents for the next generation.

Centre: Tournament selection — a random subset of individuals competes and the winner (lowest objective) becomes a parent. **Right:** Roulette wheel — each bar height represents the selection probability of one individual, proportional to its fitness.

9.2.1 Selection: Choosing Which Individuals Reproduce II

How to Read the Roulette-Wheel Formula

Imagine drawing a roulette wheel where each individual i gets a slice of the wheel whose width is proportional to its fitness score. Spinning the wheel and seeing where it lands is equivalent to sampling from the probability distribution $P(\text{select } i) \propto \text{fitness}_i$. An individual with twice the fitness of another gets twice as wide a slice and is twice as likely to be selected. This is the most “theoretically natural” formulation of selection pressure, but it can be unstable when one individual is far fitter than all others: it then dominates the wheel and the effective population reduces to a single individual. Tournament selection avoids this instability.

Practical Guidance: Which Method to Choose?

Truncation is simple and fast but can reduce diversity; it works well when the objective landscape is relatively smooth with few local minima. Tournament selection is the most flexible: setting $k = 2$ or $k = 3$ is a common default that provides moderate selection pressure. Roulette wheel is theoretically elegant but sensitive to scale; it is less commonly used in practice. For most engineering optimization problems, start with tournament selection with $k = 2$.

Selection: Python Implementation

```
1 import numpy as np
2
3 def truncation_select(y, k, rng):
4     """Return parent-index pairs from the top-k lowest-objective individuals."""
5     parents = np.argsort(y)[:k]          # indices of k smallest objective values
6     pairs = rng.choice(parents, (len(y), 2))
7     return pairs
8
9 def tournament_select(y, k, rng):
10    """Tournament selection: k random competitors per parent slot."""
11    def pick_one():
12        contenders = rng.choice(len(y), k, replace=False)  # k random competitors
13        return contenders[np.argmin(y[contenders])]        # winner = lowest y
14    return [(pick_one(), pick_one()) for _ in range(len(y))]
15
16 def roulette_select(y, rng):
17    """Fitness-proportional (roulette-wheel) selection."""
18    fitness = np.max(y) - y                # invert: lower y => higher fitness
19    fitness = np.maximum(fitness, 0)        # guard against numerical negatives
20    if fitness.sum() == 0:
21        fitness = np.ones_like(fitness)    # fallback: all equal if all identical
22    probs = fitness / fitness.sum()         # normalise to a probability distribution
23    idx = rng.choice(len(y), (len(y), 2), p=probs)
24    return idx
```

Listing 2: Three selection methods (Python / NumPy)

9.2.2 Crossover: Combining Parent Chromosomes I

Crossover (also called recombination) is how a genetic algorithm combines information from two parent individuals to create an offspring. The intuition is that if parent A has found good values for some genes and parent B has found good values for other genes, their offspring might inherit both sets of good values simultaneously. This is the mechanism that makes GAs qualitatively different from running many independent optimizations in parallel.

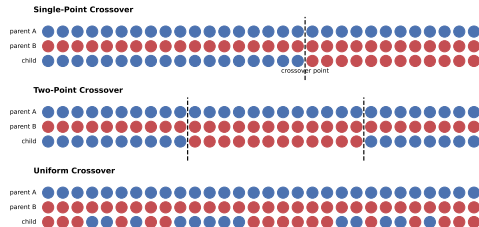
Below are four crossover schemes. The first three are for discrete or binary chromosomes; the fourth is designed for continuous (real-valued) design spaces.

Scheme 1: Single-Point Crossover

A single crossover point i is drawn uniformly at random from $\{1, \dots, n-1\}$ (1 to n minus 1), where n (n) is the chromosome length (number of design variables or genes). The offspring inherits genes 1 through i from parent A ($\mathbf{x}_a$, x sub a) and genes $i+1$ through n from parent B ($\mathbf{x}_b$, x sub b). This preserves long contiguous “building blocks” from each parent.

Example: With $n = 6$ and crossover point $i = 3$:

Parent A: [1, 4, 2 | 7, 3, 5]



Blue segments are genes inherited from parent A; red segments are genes inherited from parent B. **Top:** single-point crossover — one split divides the chromosome into exactly two segments. **Centre:** 15 / 48

9.2.2 Crossover: Combining Parent Chromosomes II

Notation for Interpolation Crossover

In the formula $\mathbf{x}_{\text{child}} = (1 - \lambda)\mathbf{x}_a + \lambda\mathbf{x}_b$:

- $\mathbf{x}_a$ (x sub a): the first parent's design vector.
- $\mathbf{x}_b$ (x sub b): the second parent's design vector.
- λ (lambda): the blending coefficient, typically drawn uniformly from $[0, 1]$, so that on average the offspring is the midpoint of the two parents.
- $(1 - \lambda)\mathbf{x}_a + \lambda\mathbf{x}_b$: a weighted average of the two parents, sometimes called a *convex combination* because both weights are non-negative and sum to 1.

9.2.2 Crossover: Combining Parent Chromosomes III

How to Read Interpolation Crossover

This formula says: place the offspring at a random point on the straight line segment connecting parent A to parent B in the design space. When $\lambda = 0$, the offspring is at parent A; when $\lambda = 1$, it is at parent B; when $\lambda = 0.5$, it is exactly halfway between them. This is useful for continuous problems because it smoothly explores the region between two good solutions rather than making discrete jumps. For example, if parent A is at $\mathbf{x}_a = [1, 2]$ and parent B is at $\mathbf{x}_b = [3, 6]$ and $\lambda = 0.3$, then

$$\mathbf{x}_{\text{child}} = 0.7 \cdot [1, 2] + 0.3 \cdot [3, 6] = [0.7 + 0.9, 1.4 + 1.8] = [1.6, 3.2].$$

Key Insight: Why Crossover Matters

Crossover is the mechanism that makes GAs qualitatively different from running many independent random restarts. If parent A has found good values for variables 1–5 and parent B has found good values for variables 6–10, a single-point crossover at position 5 can produce an offspring that is good at *all* variables at once. This “building block” combination is the core theoretical justification for why GAs outperform random search on structured problems.

Crossover: Python Implementation

```
1 import numpy as np
2
3 def single_point_crossover(a, b, rng):
4     """Single-point crossover of two real-valued vectors."""
5     i = rng.integers(1, len(a))          # crossover point in {1, ..., n-1}
6     return np.concatenate([a[:i], b[i:]] # first i genes from a, rest from b
7
8 def two_point_crossover(a, b, rng):
9     n = len(a)
10    i, j = sorted(rng.choice(n, 2, replace=False)) # two distinct cut points
11    return np.concatenate([a[:i], b[i:j], a[j:]] # a, then b-segment, then a
12
13 def uniform_crossover(a, b, p, rng):
14     """Each gene from parent a with probability p, else from parent b."""
15     mask = rng.random(len(a)) < p          # True where we take from a
16     return np.where(mask, a, b)
17
18 def interpolation_crossover(a, b, lam=0.5):
19     """Convex combination: offspring = (1-lambda)*a + lambda*b."""
20     return (1 - lam) * a + lam * b
```

Listing 3: Four crossover operators (Python / NumPy)

Each function takes two parent arrays *a* and *b* of equal length *n* and returns a single offspring array of the same length.

In `two_point_crossover`, `rng.choice(n, 2, replace=False)` draws two distinct integers from

9.2.3 Mutation: Introducing New Genetic Material I

Mutation is the third and final operator in a genetic algorithm. Even with selection and crossover operating correctly, a GA can only explore combinations of gene values that were already present in the initial population. If the global optimum requires a gene value that no individual in the initial population possesses, crossover alone can never produce it — because crossover only mixes existing gene values, it cannot create new ones.

Mutation solves this problem by randomly perturbing individual genes, ensuring that new gene values can always be discovered throughout the run, no matter what values were present at initialisation.

Distribution Mutation — Algorithm 9.5

Each gene v (v , a real number representing one design variable) in the offspring chromosome is mutated independently with probability λ (lambda, the *mutation rate*). When a gene is selected for mutation, a random perturbation ε (epsilon) drawn from a probability distribution $\mathcal{D}$ is added to it:

$$v \leftarrow v + \varepsilon, \quad \varepsilon \sim \mathcal{D}$$

where $\mathcal{D}$ is typically the Gaussian (normal) distribution $\mathcal{N}(0, \sigma^2)$ (“N of 0 and sigma squared”), meaning the

Gaussian Mutation (Python)

```
def gaussian_mutate(child, sigma,
                    lam=None, rng=None):
    rng = rng or np.random.default_rng()
    n = len(child)
    lam = lam or 1.0 / n
    mask = rng.random(n) < lam
    noise = rng.normal(0, sigma, n)
    return child + mask * noise
```

In this code, `mask` is a boolean array where `True` entries (occurring with probability $\lambda = 1/n$) 19 / 48

Genetic Algorithm: Complete Python Example

```
1 import numpy as np
2
3 def genetic_algorithm(f, bounds, pop_size=100, k_max=10,
4                       k_trunc=10, sigma=0.5, rng=None):
5     rng = rng or np.random.default_rng(0)
6     lo, hi = np.array(bounds).T
7     n = len(lo)
8     # Step 1: Initialise population uniformly in [lo, hi] per dimension
9     pop = rng.uniform(lo, hi, (pop_size, n))
10    for _ in range(k_max):
11        y = np.array([f(x) for x in pop])
12        # Selection: truncation --- keep best k_trunc individuals
13        parents = pop[np.argsort(y)[:k_trunc]]
14        new_pop = []
15        while len(new_pop) < pop_size:
16            a_idx, b_idx = rng.choice(k_trunc, 2, replace=False)
17            # Crossover: single-point at a random position
18            cp = rng.integers(1, n) # crossover point
19            child = np.concatenate([parents[a_idx, :cp],
20                                   parents[b_idx, cp:]])
21            # Mutation: Gaussian with rate lambda = 1/n
22            mask = rng.random(n) < (1.0 / n)
23            child += mask * rng.normal(0, sigma, n)
24            child = np.clip(child, lo, hi) # enforce bounds
25            new_pop.append(child)
26        pop = np.array(new_pop)
27    y = np.array([f(x) for x in pop])
```

Genetic Algorithm: Complexity and Design Choices I

Per-Generation Computational Cost

Step	Cost
Evaluate f at all m individuals	$O(m \cdot T_f)$
Sort population for selection	$O(m \log m)$
Generate m children (crossover)	$O(m \cdot n)$
Mutate all children	$O(m \cdot n)$
Total per generation	$O(m(T_f + n \log m))$

Here m (m) is the population size, n (n) is the number of design variables (chromosome length), T_f (T sub f) is the cost of one evaluation of f , and O (big- O notation) means “proportional to, up to a constant factor”.

When T_f dominates (as it does for expensive simulations such as finite-element analysis or fluid dynamics), the total cost per generation is simply $O(m \cdot T_f)$ — each generation costs m function evaluations.

Four Key Design Choices

1. Selection pressure. Using a small truncation size k (e.g. $k = 5$ out of $m = 100$, meaning the top 5%) applies strong selection pressure. This leads to fast convergence but can cause premature loss of diversity. Use a larger k (weaker pressure, say $k = 20\%$) when the landscape has many local minima and you want to preserve diverse breeding stock.

2. Mutation rate. The default $\lambda = 1/n$ (one mutation per chromosome on average) is a good starting point. If the algorithm gets stuck at the same local minimum repeatedly, increase λ to $3/n$ or $5/n$ to encourage more exploration. If it is finding a good region but failing to refine it, decrease λ or reduce σ .

3. Elitism. Always carry the current best

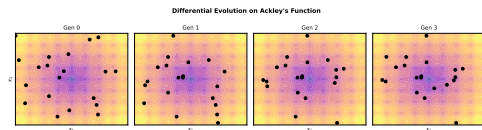
Differential Evolution: Core Idea and Algorithm I

Differential Evolution (DE) was introduced by Storn and Price in 1997. It shares the population structure of GAs but uses a fundamentally different mechanism to generate new candidate solutions. Instead of combining parents via discrete crossover, DE uses the *difference* between two randomly chosen population members to construct a perturbation and adds it to a third individual (the “anchor”). This makes DE particularly effective on continuous design spaces because the perturbation size automatically adapts to the current spread of the population: when individuals are far apart, perturbations are large (broad exploration); when individuals have converged, perturbations are small (fine-tuning).

DE/rand/1/bin — Algorithm 9.6

The name “DE/rand/1/bin” means: Differential Evolution, random anchor, 1 difference vector, binomial crossover. For each individual $\mathbf{x}^{(i)}$ (x superscript i) in the population, execute:

- 1 Sample three distinct individuals **a** (a), **b** (b), **c** (c) from the population, all different from each other and from $\mathbf{x}^{(i)}$.
- 2 Form a *trial vector* (also called the mutant vector) using a random anchor plus a scaled difference:



DE applied to Ackley's function (a highly multi-modal benchmark with many local minima and one global minimum at the origin) with parameters $p = 0.5$ and $w = 0.5$. The population (white dots) progressively concentrates near the global minimum as iterations proceed.

Differential Evolution: Core Idea and Algorithm II

How to Read the Trial Vector Formula

The formula $\mathbf{z} = \mathbf{a} + w(\mathbf{b} - \mathbf{c})$ has a geometric interpretation. Start at individual $\mathbf{a}$ (the anchor). The quantity $\mathbf{b} - \mathbf{c}$ is a vector pointing from $\mathbf{c}$ to $\mathbf{b}$ in the design space. Scaling by w and adding to $\mathbf{a}$ moves us from $\mathbf{a}$ in the direction of $\mathbf{b}$ relative to $\mathbf{c}$.

For example, in 2D: if $\mathbf{a} = [1, 2]$, $\mathbf{b} = [4, 5]$, $\mathbf{c} = [2, 3]$, and $w = 0.5$, then:
 $\mathbf{z} = [1, 2] + 0.5 \cdot ([4, 5] - [2, 3]) = [1, 2] + 0.5 \cdot [2, 2] = [1, 2] + [1, 1] = [2, 3]$. The trial vector is midway between $\mathbf{a}$ and the point you would reach by following the difference $\mathbf{b} - \mathbf{c}$ from $\mathbf{a}$.

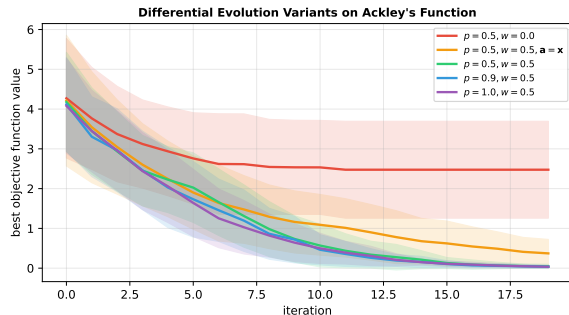
Key Insight: Self-Adapting Step Size

The term $w(\mathbf{b} - \mathbf{c})$ is the “difference vector”. Its magnitude automatically scales with the current diversity of the population: when the population is spread out (early in the search), $\|\mathbf{b} - \mathbf{c}\|$ (the Euclidean distance between $\mathbf{b}$ and $\mathbf{c}$) is large and perturbations are large, enabling broad exploration. As the population converges, $\|\mathbf{b} - \mathbf{c}\|$ shrinks naturally, and perturbations become small, enabling fine-tuning. This self-adaptation of step size is a key strength of DE over methods that use a fixed σ (sigma) throughout.

Differential Evolution: Python Implementation

```
1 import numpy as np
2
3 def differential_evolution(f, bounds, pop_size=20, k_max=200,
4                             p=0.5, w=0.5, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     n = len(lo)
8     pop = rng.uniform(lo, hi, (pop_size, n))    # uniform initialisation
9
10    for _ in range(k_max):
11        new_pop = pop.copy()
12        for i in range(pop_size):
13            # Step 1: pick 3 distinct individuals != i
14            choices = [j for j in range(pop_size) if j != i]
15            a, b, c = pop[rng.choice(choices, 3, replace=False)]
16            # Step 2: form trial vector  $z = a + w(b - c)$ 
17            interim = a + w * (b - c)
18            # Step 3: binomial crossover, guarantee at least one component
19            mask = rng.random(n) < p
20            if not mask.any():
21                mask[rng.integers(n)] = True    # force at least one change
22            child = np.where(mask, interim, pop[i])
23            child = np.clip(child, lo, hi)
24            # Step 4: greedy acceptance --- keep whichever is better
25            if f(child) < f(pop[i]):
26                new_pop[i] = child
27    pop = new_pop
```

Differential Evolution: Parameter Analysis and Variants I



Five DE variants tested on Ackley's function (1000 independent trials, population size $m = 20$, 2-dimensional problem). Each curve shows the mean best objective value ± 1 standard deviation (σ , sigma) over iterations. Lower is better; we want the curve to reach zero (the global minimum) quickly.

Key Findings from Variant Comparison

The five variants differ only in their choice of parameters p and w , or in how the anchor $\mathbf{a}$ is chosen.

Case 1: $w = 0$ (no differential perturbation).

When $w = 0$, the trial vector simplifies to $\mathbf{z} = \mathbf{a} + 0 \cdot (\mathbf{b} - \mathbf{c}) = \mathbf{a}$. No new gene values outside the current population's range can ever be generated, because $\mathbf{z}$ is always one of the existing population members. The algorithm stagnates immediately — this is the worst-performing variant.

Case 2: $\mathbf{a} = \mathbf{x}^{(i)}$ (perturb around self). Using the current individual as the anchor biases the search toward local exploration around each individual's own position. This is called DE/current/1/bin and is harder to escape local

Differential Evolution: Parameter Analysis and Variants II

Roles of the Three Key Components

The differential weight w controls the exploration radius: set w too small and the algorithm cannot escape the current cluster of individuals; set w too large and the algorithm jumps around randomly without refining. The anchor $\mathbf{a}$ determines where the perturbation is centred: using a random individual (DE/rand) encourages diversity, while using the current best (DE/best) encourages faster but riskier convergence. The crossover probability p controls how many dimensions of the current individual are replaced by the trial vector, allowing you to tune the effective step size dimension by dimension.

Particle Swarm Optimization: Inspiration and Update Equations I

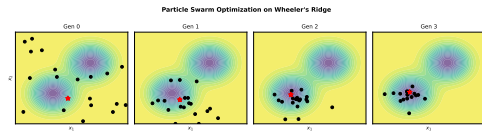
Particle Swarm Optimization (PSO) was introduced by Kennedy and Eberhart in 1995, inspired by the collective movement of bird flocks and fish schools. The key observation is that animals in a group coordinate their movement using two types of information: their own personal experience (where they personally found food before) and the collective knowledge of the group (where the best food location known to *anyone* in the group is).

In PSO, each individual is called a *particle*. Unlike GA or DE individuals — which only have a position — each particle also has a *velocity* that carries momentum from one iteration to the next. This velocity connects adjacent time steps and allows the algorithm to build up directed movement toward promising regions.

PSO State: What Each Particle Stores

Each particle i maintains three quantities simultaneously:

- $\mathbf{x}^{(i)}$ (x superscript i): the particle's current position in the design space — the current candidate solution being represented by this particle.
- $\mathbf{v}^{(i)}$ (v superscript i): the particle's current velocity, a vector of the same dimension as $\mathbf{x}$ that represents how far and in what direction the particle moves each



PSO applied to Wheeler's Ridge (a multi-modal benchmark function). The red star marks the global best position found so far. Particle arrows show their current velocities. The swarm identifies two competing modes initially; it eventually identifies the true global minimum.

Particle Swarm Optimization: Inspiration and Update Equations II

How to Read the Velocity Update

The velocity update (Eq. 1 above) has three additive terms.

The first term $w\mathbf{v}$ is *inertia*: it says “keep moving in the direction you were already going, but slow down slightly (multiply by $w < 1$)”.

The second term $c_1 r_1 \odot (\mathbf{x}_{\text{best}} - \mathbf{x})$ is the *cognitive pull*: a force directed from the particle’s current position toward its personal best position. It is proportional to the distance $\mathbf{x}_{\text{best}} - \mathbf{x}$: stronger when the particle is far from its personal best, weaker when it is close.

The third term $c_2 r_2 \odot (\mathbf{x}^* - \mathbf{x})$ is the *social pull*: a force directed toward the global best. It is strongest when the particle is far from the global best.

Typical values: $w = 0.1$, $c_1 = 0.25$, $c_2 = 2.0$ (strong social pull relative to cognitive) or $w = 0.9$, $c_1 = c_2 = 2.0$ (more exploratory with high inertia).

PSO: Python Implementation

```
1 import numpy as np
2
3 def particle_swarm(f, bounds, n_particles=30, k_max=100,
4                   w=0.1, c1=0.25, c2=2.0, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     n = len(lo)
8     # Initialise positions uniformly in [lo, hi] and velocities near zero
9     pos = rng.uniform(lo, hi, (n_particles, n))
10    vel = rng.normal(0, 0.5, (n_particles, n)) # small initial velocities
11    x_best = pos.copy() # personal bests = initial positions
12    y_best = np.array([f(p) for p in pos]) # personal best objective values
13    g_best = x_best[np.argmin(y_best)].copy() # global best position
14
15    for _ in range(k_max):
16        r1 = rng.random((n_particles, n)) # random matrices (re-sampled each iter)
17        r2 = rng.random((n_particles, n))
18        # Velocity update: inertia + cognitive pull + social pull
19        vel = (w * vel
20              + c1 * r1 * (x_best - pos) # cognitive term
21              + c2 * r2 * (g_best - pos)) # social term
22        pos = np.clip(pos + vel, lo, hi) # move particles, enforce bounds
23        y = np.array([f(p) for p in pos])
24        # Update personal bests for any particle that improved
25        improved = y < y_best
26        x_best[improved] = pos[improved]
27        y_best[improved] = y[improved]
```

PSO: Intuition, Convergence, and Comparison with GA I

Three Forces Acting on Each Particle

Force 1: Inertia ($w\mathbf{v}$). Each particle carries momentum from its previous velocity. This prevents particles from stopping abruptly when the cognitive and social forces cancel out, and allows them to overshoot an attractor and explore beyond it. When w is large (close to 1), particles travel in long sweeping arcs; when w is small (close to 0), they change direction rapidly in response to the other two forces.

Force 2: Cognitive pull ($c_1 r_1(\mathbf{x}_{\text{best}} - \mathbf{x})$). This pulls the particle toward the best position it has personally visited. It acts as individual memory: even if the particle drifts away from a good region due to social pressure, this force brings it back. This prevents the entire swarm from blindly chasing a single attractor.

Force 3: Social pull ($c_2 r_2(\mathbf{x}^* - \mathbf{x})$). This pulls every particle toward the best position found by any particle

Computational Complexity

Per iteration: $O(m \cdot T_f + m \cdot n)$. Storage: $O(m \cdot n)$ for positions, velocities, and personal bests. Here m (m) is the number of particles, n (n) is the number of design variables, and T_f (T sub f) is the cost of one function evaluation. This linear cost in both m and n is the same as DE and much cheaper than the firefly algorithm's $O(m^2)$ cost.

PSO vs. Genetic Algorithms: Key Differences

Property	GA	PSO
Individual memory	no	yes (velocity + personal best)
Crossover	yes	no
Mutation	yes	inertia plays a similar role

Firefly Algorithm: Brightness-Based Attraction I

The Firefly Algorithm was proposed by Xin-She Yang in 2009. It is inspired by the bioluminescent signalling behaviour of fireflies: each firefly emits light, and less-bright fireflies are attracted toward brighter ones. In the optimization context, the *brightness* of an individual is inversely related to its objective value — a lower $f(\mathbf{x}^{(i)})$ makes individual i brighter (more attractive). Individuals with higher objective values (dimmer fireflies) move toward individuals with lower objective values (brighter fireflies), so the whole population gradually concentrates near the best solutions found.

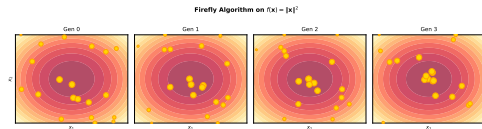
Firefly Algorithm Update — Algorithm 9.8

At each iteration, for every ordered pair of individuals (i, j) such that $f(\mathbf{x}^{(j)}) < f(\mathbf{x}^{(i)})$ (meaning individual j is brighter than individual i):

- 1 Compute the Euclidean distance between the two individuals:

$$r = \left\| \mathbf{x}^{(i)} - \mathbf{x}^{(j)} \right\|$$

where r (r) is the non-negative distance scalar, and $\| \cdot \|$ (double-bar norm) denotes the Euclidean distance, i.e., the straight-line distance in the



The firefly algorithm minimising $f(\mathbf{x}) = \|\mathbf{x}\|^2$. Dot size encodes brightness (larger dots = brighter = lower f). Over iterations, all fireflies concentrate near the global minimum at the origin as dimmer individuals are attracted toward brighter ones.

Firefly Algorithm: Brightness-Based Attraction II

How to Read the Attraction Formula

The formula $\beta(r) = \beta_0 e^{-\gamma r^2}$ says: attraction starts at β_0 when two fireflies are at the same point ($r = 0$) and decreases to zero as they move apart.

For example, with $\beta_0 = 1$ and $\gamma = 1$:

- At $r = 0$: $\beta = e^0 = 1$ (full attraction).
- At $r = 1$: $\beta = e^{-1} \approx 0.37$ (moderate attraction).
- At $r = 2$: $\beta = e^{-4} \approx 0.018$ (very weak attraction).
- At $r = 3$: $\beta = e^{-9} \approx 0.0001$ (essentially zero).

This means that nearby fireflies attract strongly and form tight local clusters, while distant fireflies are nearly independent. The algorithm effectively forms multiple sub-populations around the best individuals, making it well-suited to multi-modal landscapes.

Firefly Algorithm: Python Implementation

```
1 import numpy as np
2
3 def firefly(f, bounds, n=20, k_max=100,
4           alpha=0.3, beta0=1.0, gamma=1.0, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     dim = len(lo)
8     pop = rng.uniform(lo, hi, (n, dim))    # uniform initialisation
9
10    for _ in range(k_max):
11        y = np.array([f(p) for p in pop])
12        new_pop = pop.copy()
13        for i in range(n):
14            for j in range(n):
15                if y[j] < y[i]:                # j is brighter (lower objective)
16                    r = np.linalg.norm(pop[i] - pop[j])    # Euclidean distance
17                    beta = beta0 * np.exp(-gamma * r**2)    # attraction strength
18                    # Move i toward j with attraction beta, plus random walk alpha
19                    new_pop[i] = (new_pop[i]
20                                   + beta * (pop[j] - new_pop[i])
21                                   + alpha * rng.normal(0, 1, dim))
22        pop = np.clip(new_pop, lo, hi)
23
24    y = np.array([f(p) for p in pop])
25    return pop[np.argmin(y)]
26
27 # Note:  $O(n^2 * T_f)$  per iteration --- quadratic in population size!
```

Cuckoo Search: Levy Flights and Nest Abandonment I

Cuckoo Search was proposed by Xin-She Yang and Suash Deb in 2009. It is inspired by the brood-parasitism behaviour of cuckoo birds, which lay their eggs in the nests of other bird species. The mapping to optimization is: each cuckoo's egg is a candidate solution (a design vector $\mathbf{x}$), and nests represent locations in the design space. A nest with a high-quality egg (low objective value) survives to the next generation; nests with poor eggs may be discovered and abandoned.

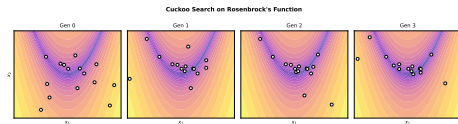
The key technical innovation of cuckoo search is its use of *Levy flights* for exploration. A Levy flight is a type of random walk in which step lengths follow a heavy-tailed distribution (approximated here by the Cauchy distribution), enabling occasional very large jumps that help escape local minima.

Cuckoo Search Algorithm — Algorithm 9.9

The algorithm maintains a population of m (m) nests (candidate solutions), sorted by objective value.

- 1 **Levy flight step.** For each of m_s (m sub s) nests chosen in sequence: pick a random nest index j , generate a new candidate via a Cauchy (heavy-tailed) flight step scaled by the bounds range:

$$\mathbf{x}' = \mathbf{x}^{(j)} + \text{Cauchy}(r) \cdot (\mathbf{u} - \mathbf{l})$$



Cuckoo search applied to Rosenbrock's function $f(\mathbf{x}) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$, whose global minimum is at $(1, 1)$ where $f = 0$. The nests (white dots) gradually concentrate near $(1, 1)$ as the algorithm progresses. Rosenbrock's function is notoriously difficult because its minimum lies in a narrow, curved valley.

Cuckoo Search: Levy Flights and Nest Abandonment II

Why Cauchy Flights Enable Better Global Exploration

A standard Gaussian random walk produces step sizes that are almost always small, because the tails of the Gaussian distribution decay very rapidly (proportional to e^{-x^2}). The Cauchy distribution has *heavy tails*: they decay only proportionally to $1/x^2$, which is much slower. This means: most iterations, the algorithm makes a small local refinement step (the peak of the Cauchy distribution is near zero) — but occasionally it makes a very large jump that can cross an entire valley or escape a basin of attraction. This mix of predominantly local steps with rare long-range jumps is called a Levy flight pattern and is an efficient strategy for complex, multi-modal landscapes.

Cuckoo Search: Python Implementation

```
1 import numpy as np
2
3 def cuckoo_search(f, bounds, pop_size=25, k_max=100,
4                   p_a=0.25, m_search_frac=1.0, r=0.01, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     n = len(lo)
8     m_search = max(1, int(m_search_frac * pop_size)) # nests that receive a step
9     m_abandon = max(1, int(p_a * pop_size))          # nests to abandon each iter
10
11     # Initialise: list of (x, y) pairs sorted by objective value ascending
12     pop = [(x, f(x)) for x in rng.uniform(lo, hi, (pop_size, n))]
13     pop.sort(key=lambda t: t[1])
14
15     for _ in range(k_max):
16         # Step 1: Levy (Cauchy) flight for m_search nests
17         for i in range(m_search):
18             j = rng.integers(pop_size) # pick random nest
19             step = rng.standard_cauchy(n) * r * (hi - lo) # Cauchy step scaled by range
20             x_new = np.clip(pop[j][0] + step, lo, hi)
21             y_new = f(x_new)
22             if y_new < pop[i][1]: # greedy acceptance
23                 pop[i] = (x_new, y_new)
24         # Step 2: Sort and abandon the m_abandon worst nests
25         pop.sort(key=lambda t: t[1])
26         for i in range(pop_size - m_abandon, pop_size):
27             x_new = rng.uniform(lo, hi, n) # random replacement
```

Cuckoo Search: Key Properties and Parameter Summary I

Cauchy vs. Gaussian Flights: A Comparison

The Cauchy distribution (used to approximate Levy flights) has the following properties relevant to optimization:

- Its probability density function has *heavier tails* than the Gaussian (normal) distribution. Where a Gaussian step of size 10σ is astronomically unlikely (probability $\sim 10^{-23}$), a Cauchy step of the same size is merely uncommon (probability $\sim 1/100$).
- Most Cauchy-distributed steps are small (the peak of the distribution is at zero), providing local refinement in most iterations.
- Occasionally, a very large step occurs — a “Levy jump” — which can move the algorithm across a large basin to explore a completely different

Computational Complexity

Step	Cost per iteration
m_s Levy flights	$O(m_s \cdot T_f)$
Sort nests	$O(m \log m)$
Abandon and replace m_a nests	$O(m_a \cdot T_f)$
Total per iteration	$O(m \cdot T_f)$

The overall cost is linear in population size m — the same order as PSO and DE, and much cheaper than the firefly algorithm's $O(m^2 T_f)$ (m squared times T sub f).

Parameter Summary and Typical Values

Parameter	Meaning	Typical value
m	population size	15–50

9.7 Hybrid Methods: Combining Global and Local Search I

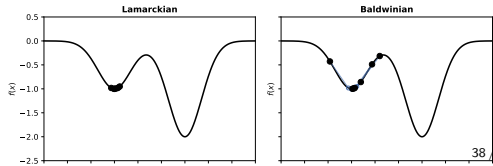
Population methods are excellent at *global exploration*: they maintain diversity across the design space and can locate the basin of attraction of the global minimum even in highly multi-modal landscapes. However, they are often *slow at refining* a solution once they are near a minimum — they approach the optimum gradually over many generations, whereas a dedicated local search method (such as gradient descent or Nelder-Mead) would converge precisely in far fewer evaluations.

Local search methods (gradient descent, Nelder-Mead simplex, etc.) are the opposite: they converge quickly to a precise local minimum near their starting point, but they have no mechanism for escaping local minima if they start far from the global optimum.

Hybrid methods combine both strategies: use a population method for global exploration to locate promising basins, and a local search method for local refinement to exploit those basins efficiently. The key design question is: should the local search move the individual to its local minimum (updating its position), or should it only be used to compute a better fitness value for selection purposes while the individual stays at its original position?

Philosophy 1: Lamarckian Learning

Named after the French biologist Jean-Baptiste Lamarck (1744–1829), who proposed that organisms can pass on traits acquired during their lifetime directly to their



9.7 Hybrid Methods: Combining Global and Local Search II

Key Takeaway: When to Use Each Philosophy

If the problem has few local minima and you need fast precise convergence, Lamarckian learning is effective and straightforward. If the landscape is complex with many local minima and you are unsure whether the current population is near the global optimum, use Baldwinian learning to preserve the population's ability to explore multiple basins simultaneously. As a general rule of thumb: when in doubt, use Baldwinian — it costs no more than Lamarckian and is safer on complex landscapes.

Baldwinian Hybrid GA: Python Example

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def baldwinian_ga(f, bounds, pop_size=30, k_max=50,
5                  k_trunc=8, sigma=0.3, rng=None):
6     rng = rng or np.random.default_rng()
7     lo, hi = np.array(bounds).T
8     n = len(lo)
9     pop = rng.uniform(lo, hi, (pop_size, n))
10
11     for _ in range(k_max):
12         # Baldwinian: use local-search VALUE for selection, keep original POSITION
13         def local_val(x):
14             res = minimize(f, x, method='Nelder-Mead',
15                           options={'maxiter': 20, 'xatol': 1e-4})
16             return res.fun # return improved objective value, NOT the improved position
17
18         y = np.array([local_val(x) for x in pop]) # pop positions are unchanged
19
20         # Standard GA: truncation selection + single-point crossover + Gaussian mutation
21         parents = pop[np.argsort(y)[:k_trunc]]
22         new_pop = []
23         while len(new_pop) < pop_size:
24             a, b = parents[rng.choice(k_trunc, 2, replace=False)]
25             cp = rng.integers(1, n)
26             child = np.concatenate([a[:cp], b[cp:]])
27             child += rng.normal(0, sigma, n) * (rng.random(n) < 1/n)
```

Lamarckian vs. Baldwinian: Worked Example and Comparison I

Worked Example: A Two-Basin Landscape

Consider the one-dimensional function:

$$f(x) = -e^{-x^2} - 2e^{-(x-3)^2}$$

where e (Euler's number, approximately 2.718) is the base of the natural exponential function. This function has *two local minima*: a shallower one near $x = 0$ (where $f(0) = -1$) and the global minimum near $x = 3$ (where $f(3) \approx -2$, meaning it is twice as deep as the local minimum).

Suppose we start with a population of 10 individuals, all clustered near $x = 0$.

Under Lamarckian learning: each individual near $x = 0$ is moved by local search to the local minimum at $x \approx 0$. All 10 individuals now sit at the local minimum. All future crossover between these identical individuals produces offspring also near

Comparison Table

Property	Lamarckian	Baldwinian
Exploits local info	Strongly	Via selection o
Preserves diversity	No	Yes
Convergence speed	Fast locally	Moderate
Global reach	Low (gets trapped)	Higher
Extra cost per gen.	$O(T_{\text{local}})$	$O(T_{\text{local}})$
Risk of local trap	High	Low

In this table, T_{local} (T sub local) is the cost of running the local optimizer on one individual. With Nelder-Mead limited to 20 iterations, T_{local} is approximately 20 times the cost of a single objective evaluation. Both methods pay this same overhead, but Baldwinian uses it more wisely.

Both methods incur the same additional cost per individual per generation: running a local optimizer that performs T_{local} evaluations. Since Nelder-Mead or a few gradient steps are typically

Chapter 9 Summary I

This chapter introduced the family of *population methods* for optimization — algorithms that maintain a collection of m (m) candidate solutions (a “population”) and evolve them collectively over many iterations called generations. Unlike single-point methods (gradient descent, Nelder-Mead), population methods share information across the design space, making them much more robust to local minima on complex, multi-modal objectives.

Genetic Algorithms (Section 9.2)

GAs are biologically inspired. Three operators are applied each generation: (1) **Selection** converts objectives to fitness scores and biases reproduction toward better individuals. (2) **Crossover** combines gene sequences from two parents to create offspring, mixing good gene combinations from different individuals. (3) **Mutation** introduces random perturbations so that new gene values can be discovered beyond those present in the initial population. GAs work naturally on both discrete (binary, integer) and continuous design spaces.

Firefly Algorithm (Section 9.5)

The firefly algorithm models pairwise attraction based on brightness (fitness): dimmer individuals move toward brighter ones with attraction strength $\beta(r) = \beta_0 e^{-\gamma r^2}$ (beta-zero times e to the minus gamma r squared) that decays exponentially with distance r (r), creating natural local clustering. Main limitation: $O(m^2)$ cost per iteration.

Cuckoo Search (Section 9.6)

Cuckoo search uses heavy-tailed Cauchy (Levy)

Method Comparison: Complexity and Practical Guidance I

Algorithm Complexity and Parameter Summary

Method	Per-iteration cost	Key parameters	Strengths	Best for
Genetic Algorithm	$O(m(T_f + n \log m))$	m, k, λ, σ	Flexible encoding	Discrete + continuous
Differential Evolution	$O(m \cdot T_f)$	m, p, w	Self-adapting step size	Continuous, multi-modal
PSO	$O(m \cdot T_f)$	m, w, c_1, c_2	Fast, simple	Continuous, fast conv.
Firefly	$O(m^2 T_f)$	$m, \alpha, \beta_0, \gamma$	Pairwise attraction	Multi-modal, small m
Cuckoo Search	$O(m \cdot T_f)$	m, p_a, r	Long-range jumps	Rugged, continuous

In this table: m (m) is population size, T_f (T sub f) is cost of one objective evaluation, n (n) is number of design variables, λ (λ) is mutation rate, σ (σ) is mutation standard deviation, w (w) is differential weight or inertia, c_1 (c sub 1) is cognitive coefficient, c_2 (c sub 2) is social coefficient, α (α) is random walk step, β_0 (β sub 0) is max attractiveness, γ (γ) is light absorption, p_a (p sub a) is abandonment probability, r (r) is Cauchy scale.

Method Comparison: Complexity and Practical Guidance II

Practical Guidance: Which Method to Use?

Unknown landscape — first attempt. Start with PSO or DE. Both have linear cost in m , few parameters, and are robust across a wide range of continuous problems. PSO is particularly easy to tune (adjust w to control exploration vs. exploitation).

Mixed-integer or combinatorial problem. Use a GA with an appropriate encoding: binary genes with bit-flip mutation for combinatorial problems, or real-valued genes with Gaussian mutation for continuous variables.

Highly multi-modal landscape. Cuckoo search (Levy flights enable long-range jumps across basins) or the firefly algorithm (pairwise attraction can maintain multiple sub-populations around different modes simultaneously) are good choices.

Need fast local refinement after global search. Wrap any population method with Baldwinian (preferred) or Lamarckian local search. Use a lightweight local optimizer such as Nelder-Mead (derivative-free) or L-BFGS (gradient-based if derivatives are available).

Limited evaluation budget (expensive f). Population size m is the primary lever: a smaller m uses fewer evaluations per generation but may miss the global optimum. With a budget of B total evaluations, set m and $k_{\max}$ so that $m \times k_{\max} = B$. Larger m with fewer generations favours exploration; smaller m with more generations favours exploitation.

Selected Exercises with Worked Solutions I

9.1 *What is the purpose of the selection operation in genetic algorithms?*

Solution. Selection serves to bias reproductive success toward individuals with lower objective values (higher fitness), ensuring that the gene pool of the next generation is dominated by better-performing individuals. Without selection, the GA would perform random crossover and mutation with no pressure toward improvement — it would behave like a random search. Selection is the mechanism by which useful genetic “building blocks” (combinations of gene values that contribute to low objective values) are preserved and amplified across generations. Without selection, good gene combinations appear randomly by chance just as often as bad ones, and the population does not converge.

9.2 *Why does mutation play a fundamental role in GAs? If you suspect there is a better optimum elsewhere, how should you adjust λ (lambda)?*

Solution. Mutation is fundamental because crossover alone can only combine gene values that already exist in the population. If the optimal solution requires a gene value that was not present in any individual at initialisation — for example, if the global optimum requires $x_1 = 7.3$ but all initial individuals have $x_1 \in [-5, 0]$ — crossover can *never* discover it. Mutation introduces new gene values by random perturbation (adding $\varepsilon \sim \mathcal{N}(0, \sigma^2)$), enabling the GA to explore regions not represented in the initial population.

Selected Exercises with Worked Solutions II

If a better optimum is suspected elsewhere (for example, the best individual has not improved for many consecutive generations), *increase* λ above the default $1/n$ — for example, to $3/n$ or $5/n$ — to generate more exploration per generation at the cost of potentially disrupting near-optimal solutions.

9.3 *PSO converges to a non-global minimum. How can you adjust the parameters?*

Solution. Premature convergence in PSO usually means the social force $c_2 r_2 (\mathbf{x}^* - \mathbf{x})$ is overwhelming all particles toward the current global best, which happens to be a local (not global) minimum. Three remedies: (1) Increase the number of particles $n_{\text{particles}}$ so the initial population is more spread out and more likely to have at least one particle near the true global minimum. (2) Increase the cognitive coefficient c_1 relative to c_2 , strengthening each particle's attraction to its own personal best (individual memory) and weakening the social pull. (3) Increase the inertia weight w to give particles more momentum to overshoot the local attractor and explore beyond it. If the problem is high-dimensional, consider running multiple independent PSO instances (multi-start).

Selected Exercises with Worked Solutions III

9.4 *Compare five DE variants on Ackley's function and explain the results.*

Solution. (1) $w = 0$: worst — the trial vector $\mathbf{z} = \mathbf{a} + 0 \cdot (\mathbf{b} - \mathbf{c}) = \mathbf{a}$ contains only existing population values; no new gene values outside the current range can be generated; the algorithm immediately stagnates. (2) $\mathbf{a} = \mathbf{x}^{(i)}$ (DE/current): biased toward local exploration around each individual, hard to escape local minima on highly multi-modal Ackley's function. (3) Standard $p = 0.5, w = 0.5$: balanced — a good default that performs reasonably on most problems. (4) $p = 0.9$: high crossover probability means nearly every dimension is replaced by the trial vector; performs almost as well as $p = 1.0$. (5) $p = 1.0$ (guaranteed crossover): best performer on Ackley's function — every dimension is updated at every iteration, maximising information flow from the self-adapting differential perturbation.

References I

- M. J. Kochenderfer & T. A. Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026. Chapter 9: Population Methods.
- J. H. Holland, *Adaptation in Natural and Artificial Systems*, MIT Press, 1992. (Original treatment of Genetic Algorithms and the schema theorem.)
- R. Storn & K. Price, “Differential Evolution — a simple and efficient heuristic for global optimization over continuous spaces,” *Journal of Global Optimization*, 11(4):341–359, 1997.
- J. Kennedy & R. Eberhart, “Particle Swarm Optimization,” *Proceedings of IEEE International Conference on Neural Networks (ICNN)*, pp. 1942–1948, 1995.
- X.-S. Yang, “Firefly Algorithms for Multimodal Optimization,” in *Stochastic Algorithms: Foundations and Applications (SAGA)*, Lecture Notes in Computer Science, vol. 5792, Springer, 2009.
- X.-S. Yang & S. Deb, “Cuckoo Search via Levy Flights,” in *Proceedings of the World Congress on Nature & Biologically Inspired Computing (NaBIC)*, pp. 210–214, IEEE Press, 2009.